# Phase 1 Energy: Discretization of the Γ-Convergence Functional

Dirichlet term, double well, Modica–Mortola limit, and the crispness penalty

July 2026

**Abstract**

This document derives the discretized Phase 1 relaxation energy minimized by `ProjectedGradientOptimizer` (`src/optimization/pgd_optimizer.py`) and its gradient, exactly as implemented after the double-well correction of 2026-07-08. The continuous functional is the Modica–Mortola-type Γ-convergence energy of Bogosel & Oudet: a Dirichlet interface term plus a double well, whose $\varepsilon \to 0$ limit is the partition perimeter, augmented by a soft *crispness* penalty. Three quantities are derived from the $P_1$ finite-element discretization: the Dirichlet term $\varepsilon\, u^\top K u$, the double-well term $\varepsilon^{-1} q^\top M q$ with $q = u(1-u)$, and the variance penalty $\lambda\,(1 - \mathrm{Var}/T)$; all three gradients are *analytical* and match the code to machine precision. A remark records the one-token discretization bug this document's corrected form replaces (the well was previously coded as $q = u^2(1-u)^2$; see `docs/reference/phase1_energy_discretization_bug.md`).

## Contents

# 1 Overview

Phase 1 of the method (relaxation) replaces the geometric problem "partition the surface $S$ into $N$ equal-area regions of minimal total interface length" by a smooth minimization over *density fields*. Each region $k$ is represented by a function $u_k : S \to [0,1]$, where $u_k(x)$ is "how strongly cell $k$ claims the point $x$." The optimizer minimizes a relaxed energy $E$ over the stacked field $u = (u_1, \ldots, u_N)$ subject to a partition-of-unity constraint $\sum_k u_k \equiv 1$ and an equal-area constraint $\int_S u_k \, dA = |S|/N$ for each $k$ (both enforced by projection; `src/optimization/projection.py`). As the interface width $\varepsilon \to 0$ the relaxed energy $\Gamma$-converges to the partition perimeter, so its minimizers approximate minimal-perimeter partitions [1, 2].

This document derives, from the $P_1$ finite-element discretization, the three additive pieces of $E$ and their gradients *as implemented*. It is the formal companion to the reference note `docs/reference/phase1_energy_discretization_bug.md`, which records that the double-well term was for a long time mis-discretized (a typo copied from the source paper) and was corrected on 2026-07-08; the form derived here is the corrected one. Per the `docs/math` scope policy, every formula below is a faithful transcription of the live code.

# 2 Notation and setup

**Remark 2.1** (The symbol $\lambda$)**.** In this document $\lambda$ denotes the *crispness-penalty weight* (`lambda_penalty`, Section 6). It is unrelated to the Phase 2 variable-point parameter $\lambda$ of the same name used elsewhere in `docs/math`; Phase 1 has no variable points.

**Mesh and $P_1$ finite elements.** $S$ is triangulated with vertices $\mathbf{V} = \{x_1, \ldots, x_{|V|}\}$ and triangles $\mathbf{F}$. Let $\{\varphi_j\}_{j=1}^{|V|}$ be the piecewise-linear ($P_1$) nodal "hat" basis, $\varphi_j(x_i) = \delta_{ij}$. A nodal vector $w \in \mathbb{R}^{|V|}$ defines the $P_1$ function $w_h = \sum_j w_j \varphi_j$, i.e. the linear interpolant of the values $w_j$. The two standard $P_1$ assembly matrices are the *mass* and *stiffness* matrices

$$M_{jk} \;=\; \int_S \varphi_j \varphi_k \, dA, \qquad K_{jk} \;=\; \int_S \nabla_\tau \varphi_j \cdot \nabla_\tau \varphi_k \, dA, \tag{1}$$

where $\nabla_\tau$ is the tangential (surface) gradient. Both are built by `TriMesh.compute_matrices` (`src/mesh/tri_mesh.py`); $M$ and $K$ are symmetric and sparse. The *lumped mass* vector is the row sum of $M$,

$$v_j \;=\; \sum_k M_{jk} \;=\; \int_S \varphi_j \, dA \;>\; 0, \qquad W \;=\; \sum_j v_j \;=\; \int_S 1 \, dA \;=\; |S|, \tag{2}$$

so $v_j$ is the area weight of vertex $j$ and $W$ is the total surface area (`self.v`, `self.W` in the optimizer). The equal-area target per cell is $\bar{A} = W/N$ and the target density mean is $\mu_t = 1/N$.

**Interface width.** The width is tied to the mesh, $\varepsilon = \sqrt{\text{mean triangle area}} \approx h$ (`src/pipeline/relaxation.py`, `_setup_level`), and is held fixed within a refinement level.

**The two exact $P_1$ identities.** For any nodal vector $w$,

$$\int_S |\nabla_\tau w_h|^2 \, dA \;=\; w^\top K w, \tag{3}$$

$$\int_S w_h^2 \, dA \;=\; w^\top M w, \tag{4}$$

both immediate from (1) and bilinearity, since $w_h = \sum_j w_j \varphi_j$. These are the only two discretization identities the energy uses; everything else is exact algebra.

**The continuous functional.** Per cell, the relaxed energy is the Modica–Mortola-type functional

$$F_\varepsilon(u_k) \;=\; \varepsilon \int_S |\nabla_\tau u_k|^2 \, dA \;+\; \frac{1}{\varepsilon} \int_S W\big(u_k\big) \, dA, \qquad W(s) = s^2(1-s)^2, \tag{5}$$

with the standard quartic double well $W$ (minima at $s = 0$ and $s = 1$). The total Phase 1 energy sums this over the $N$ cells and adds the penalty of Section 6:

$$E(u) \;=\; \sum_{k=1}^{N} F_\varepsilon(u_k) \;+\; \lambda \sum_{k=1}^{N} P(u_k). \tag{6}$$

## 3 The Dirichlet (interface-width) term

Discretizing $u_k$ by its $P_1$ interpolant and applying (3) gives the interface-width term exactly:

$$\varepsilon \int_S |\nabla_\tau u_k|^2 \, dA \;=\; \varepsilon \, u_k^\top K u_k \;=:\; E_k^{\mathrm{dir}}. \tag{7}$$

Because $K$ is symmetric, the gradient in the nodal unknowns $u_k$ is

$$\nabla_{u_k} E_k^{\mathrm{dir}} \;=\; 2\,\varepsilon\, K u_k. \tag{8}$$

**Code: Dirichlet term and gradient**

`src/optimization/pgd_optimizer.py`: `compute_energy` / `compute_gradient` (per-cell loop)

```
grad_term = epsilon * (u.T @ (K @ u))
grad_grad = 2 * epsilon * (K @ u)
```

## 4 The double-well term (the corrected discretization)

The interface (binarization) term is $\varepsilon^{-1} \int_S W(u_k)\, dA = \varepsilon^{-1} \int_S u_k^2 (1-u_k)^2 \, dA$. Write the integrand as a perfect square,

$$W(u_k) \;=\; u_k^2(1-u_k)^2 \;=\; \big[\, u_k(1-u_k) \,\big]^2 \;=\; q_k^2, \qquad q_k \;:=\; u_k(1-u_k). \tag{9}$$

Discretizing the *new* field $q_k$ nodally — i.e. forming the vector with entries $q_{k,j} = u_{k,j}(1-u_{k,j})$ and taking its $P_1$ interpolant — and applying the mass-matrix identity (4) gives

$$\frac{1}{\varepsilon} \int_S W(u_k)\, dA \;\approx\; \frac{1}{\varepsilon} \int_S (q_k)_h^2 \, dA \;=\; \frac{1}{\varepsilon}\, q_k^\top M q_k \;=:\; E_k^{\mathrm{well}}. \tag{10}$$

**Remark 4.1** (Nodal interpolation of the well)**.** The equality (4) is exact, but (10) carries one approximation: the integrand $q_k = u_k(1-u_k)$ is evaluated at the nodes and then interpolated linearly, so $(q_k)_h$ is the $P_1$ interpolant of the nodal products, not the exact quadratic $u_h(1-u_h)$. This is the usual nodal-quadrature treatment of a polynomial nonlinearity; it is consistent (error $O(h^2)$) and is what the code computes. Equivalently, the *discrete* double-well energy is defined as $\varepsilon^{-1} \|(q_k)_h\|_{L^2}^2$.

**Gradient.** Since $q_k = u_k \odot (1-u_k)$ acts componentwise, its Jacobian with respect to $u_k$ is diagonal:

$$\frac{\partial q_{k,j}}{\partial u_{k,j}} \;=\; 1 - 2u_{k,j} \qquad \Longrightarrow \qquad \frac{\partial q_k}{\partial u_k} \;=\; \mathrm{diag}(1-2u_k). \tag{11}$$

With $M$ symmetric, the chain rule applied to $q_k^\top M q_k$ gives

$$\nabla_{u_k} E_k^{\text{well}} \;=\; \frac{1}{\varepsilon}\,\operatorname{diag}(1-2u_k)\,\big(2Mq_k\big) \;=\; \frac{2}{\varepsilon}\,\big(Mq_k\big)\odot(1-2u_k), \tag{12}$$

where $\odot$ is the Hadamard (elementwise) product.

**Code: double-well term and gradient (corrected)**

`src/optimization/pgd_optimizer.py`: `compute_energy` / `compute_gradient` (per-cell loop)

```
interface_vec = u * (1 - u)                                  (= q_k; not u**2*(1-u)**2)
interface_term = (1/epsilon) * (interface_vec.T @ (M @ interface_vec))
grad_interface = (2/epsilon) * (M @ interface_vec) * (1 - 2*u)
```

**Remark 4.2** (The discretization bug this form corrects)**.** Earlier code set `interface_vec = u**2 * (1-u)**2`, i.e. $q_k \mapsto q_k^2$. That transcribes a *typo* in the source paper [1] (§3), whose printed definition of the discretized field has an extra squaring, while its prose target integral $\int u^2(1-u)^2$ and its printed gradient $2Mv\odot(1-2u)$ both require $v = u(1-u)$. The buggy energy computed $\varepsilon^{-1}\int (u^2(1-u)^2)_h^2 = \varepsilon^{-1}\int u^4(1-u)^4$ — an eighth-degree well, $\sim 25\times$ too small in magnitude — *and* its gradient (12) was then no longer the gradient of the coded energy (finite-difference relative error $\approx 1.2$). With the correct $q_k = u_k(1-u_k)$, energy (10) and gradient (12) are mutually consistent to machine precision (FD relative error $\approx 10^{-8}$). See `docs/reference/phase1_energy_discretization_bug.md` for the full audit and the $\sim 25\times$ scale shift, which required re-tuning $\lambda$.

## 5 The Modica–Mortola limit: why this is a perimeter

The pairing in (5) is the classical Modica–Mortola functional. For a well $W \geq 0$ vanishing exactly at 0 and 1, the family $F_\varepsilon$ $\Gamma$-converges as $\varepsilon \to 0$ (in $L^1$) to

$$F_0(u) \;=\; c_W \cdot \operatorname{Per}\big(\{u=1\}\big), \qquad c_W \;=\; 2\int_0^1 \sqrt{W(s)}\,ds, \tag{13}$$

finite only on fields $u \in \{0,1\}$ of finite perimeter [2]. For the quartic well $W(s) = s^2(1-s)^2$ we have $\sqrt{W(s)} = s(1-s)$ on $[0,1]$, so

$$c_W \;=\; 2\int_0^1 s(1-s)\,ds \;=\; 2\Big[\tfrac{1}{2}-\tfrac{1}{3}\Big] \;=\; \tfrac{1}{3}. \tag{14}$$

Summing over the $N$ cells as in (6), each interface between two adjacent regions lies in the boundary of *two* phases and is therefore counted twice, so the limit of $\sum_k F_\varepsilon(u_k)$ is $2c_W = \frac{2}{3}$ times the total length of the partition's interface network. The constant is immaterial to the minimizer: minimizing the relaxed energy at small $\varepsilon$ drives the partition toward minimal total interface length — the perimeter problem the method targets. This is the sense in which the coefficient balance $\varepsilon\, u^\top K u$ versus $\varepsilon^{-1} q^\top M q$ matters: the double-well correction of Section 4 restores $W$ to the quartic (5) whose limit constant is (14), rather than an eighth-degree well with a different (and much flatter) profile.

## 6 The crispness penalty

The double well rewards *pointwise* binarization but does not by itself force a cell to concentrate its (area-constrained) mass into a compact, sharp region. The optimizer adds a soft penalty $P$ that rewards each cell's density having the *variance of a sharp* 0/1 *indicator* of the target size. All statistics are *area-weighted* by the lumped mass $v$ of (2).

**Weighted mean and variance.** For a cell density $u$ (dropping the cell index),

$$\mu(u) \;=\; \frac{v^\top u}{W}, \qquad \mathrm{Var}(u) \;=\; \frac{1}{W}\sum_j v_j\,(u_j-\mu)^2 \;=\; \frac{\big((u-\mu\mathbf{1})\odot v\big)^\top (u-\mu\mathbf{1})}{W}. \tag{15}$$

On the constraint set $v^\top u = \bar{A}$ the mean is pinned at $\mu = \bar{A}/W = 1/N = \mu_t$.

**Target variance.** A sharp indicator equal to 1 on an area fraction $\mu_t$ and 0 elsewhere has area-weighted variance $\mu_t(1-\mu_t)$. The penalty compares $\mathrm{Var}(u)$ to a target

$$T \;=\; \begin{cases} \mu_t(1-\mu_t), & \text{fixed target (}\texttt{penalty\_target\_mode = 'fixed'}\text{)},\\ \mu(1-\mu), & \text{adaptive target (}\texttt{'adaptive'}\text{)}, \end{cases} \qquad T_{\text{eff}} = T + \varepsilon_P, \tag{16}$$

with a small floor $\varepsilon_P =$ `penalty_eps` guarding the division. The penalty per cell is

$$P(u) \;=\; \lambda\left(1 - \frac{\mathrm{Var}(u)}{T_{\text{eff}}}\right). \tag{17}$$

It is minimized (drops toward $\lambda\cdot 0$) as $\mathrm{Var}(u)\to T$, i.e. as the cell becomes as "crisp" as a sharp indicator of the right size; a diffuse, low-variance density is penalized. In the code the per-cell penalties are summed and added to $E$; the weight is $\lambda =$ `lambda_penalty`.

**Gradient.** The gradient of the weighted variance (15) is

$$\nabla_u \mathrm{Var}(u) \;=\; \frac{2}{W}\, v\odot(u-\mu\mathbf{1}), \tag{18}$$

where the terms from the $\mu$-dependence cancel: $\partial\mu/\partial u_k = v_k/W$ and $\sum_j v_j(u_j-\mu)=0$ on any $u$. For the *fixed* target $T$ is constant, so

$$\nabla_u P \;=\; -\frac{\lambda}{T_{\text{eff}}}\,\nabla_u \mathrm{Var}(u). \tag{19}$$

For the *adaptive* target $T=\mu(1-\mu)$ depends on $u$ through $\partial T/\partial u = (1-2\mu)\,v/W$, giving the quotient-rule form

$$\nabla_u P \;=\; -\lambda\left[\frac{\nabla_u \mathrm{Var}(u)}{T_{\text{eff}}} \;-\; \frac{\mathrm{Var}(u)}{T_{\text{eff}}^2}\,(1-2\mu)\,\frac{v}{W}\right]. \tag{20}$$

**Code: crispness penalty and gradient**

`src/optimization/pgd_optimizer.py`: `compute_energy` / `compute_gradient` (penalty loop)

```
mu = (v @ u) / W;  var_w = ((center * v) @ center) / W          (center = u - mu)
grad_var = (2/W) * (v * center)
fixed:    G += -lambda * (grad_var / T_eff)
adaptive: G += -lambda * (grad_var/T_eff - (var_w/T_eff**2)*(1-2*mu)*(v/W))
```

**Remark 6.1** (Operating regime after the energy fix)**.** The $\sim 25\times$ scale shift from the double-well correction (Remark 4.2) rebalances $\lambda$ against the well and Dirichlet terms. Empirically, on the corrected energy $\lambda$ becomes an effective lever where it was inert before: a moderate `lambda_penalty`$\approx 5$ resolves the high-$N$ "runt" cell that the buggy energy manufactured, and seeded initialization becomes mandatory (random initialization now stalls at the symmetric state). Both effects are measured in `docs/experiments/02-corrected-energy-highn-validation/`; they are consequences of the energy *scale*, not of the formulas derived here.

## 7 Assembled energy and gradient

Collecting Sections 3–6, with $q_k = u_k \odot (1 - u_k)$, the discretized Phase 1 energy and its gradient (column $k$ acting on cell $k$) are

$$E(u) \;=\; \sum_{k=1}^{N} \Big[\, \varepsilon\, u_k^\top K u_k \;+\; \tfrac{1}{\varepsilon}\, q_k^\top M q_k \,\Big] \;+\; \lambda \sum_{k=1}^{N} \Big(1 - \tfrac{\mathrm{Var}(u_k)}{T_{\mathrm{eff}}}\Big), \tag{21}$$

$$\nabla_{u_k} E \;=\; \underbrace{2\varepsilon\, K u_k}_{\text{Dirichlet}} \;+\; \underbrace{\tfrac{2}{\varepsilon}\,(M q_k) \odot (1 - 2u_k)}_{\text{double well}} \;+\; \underbrace{\nabla_{u_k} P(u_k)}_{(19)\text{ or }(20)}. \tag{22}$$

The gradient (22) is the unconstrained (Euclidean) gradient the optimizer then projects onto the tangent of the sum-to-one and equal-area constraints (`src/optimization/projection.py`); the constraint handling is outside the scope of this energy derivation.

**Code: assembled energy and gradient**

`src/optimization/pgd_optimizer.py`: `compute_energy(x, return_components)` returns {`total, grad, interface, penalty`}; `compute_gradient(x)` returns the stacked $\nabla E$ reshaped to $\mathbb{R}^{|V| \times N}$.

## Summary table

| Quantity | Discretized form | Method | Implementing function |
|---|---|---|---|
| Dirichlet term (7) | $\varepsilon\, u_k^\top K u_k$ | analytical | `compute_energy` |
| its gradient (8) | $2\varepsilon\, K u_k$ | analytical | `compute_gradient` |
| Double-well term (10) | $\varepsilon^{-1} q_k^\top M q_k,\; q_k = u_k(1 - u_k)$ | analytical | `compute_energy` |
| its gradient (12) | $\frac{2}{\varepsilon}(M q_k) \odot (1 - 2u_k)$ | analytical | `compute_gradient` |
| Weighted variance (15) | $\frac{1}{W} \sum_j v_j (u_j - \mu)^2$ | analytical | `compute_energy` |
| its gradient (18) | $\frac{2}{W} v \odot (u - \mu \mathbf{1})$ | analytical | `compute_gradient` |
| Crispness penalty (17) | $\lambda(1 - \mathrm{Var}\,/T_{\mathrm{eff}})$ | analytical | `compute_energy` |
| its gradient (fixed / adaptive) | (19) / (20) | analytical | `compute_gradient` |
| $\Gamma$-limit constant (14) | $c_W = 2\int_0^1 \sqrt{W} = \frac{1}{3}$ | analytical | — (theory) |

All gradients are analytical and consistent with the corresponding energy to machine precision (finite-difference relative error $\approx 10^{-8}$). The formulas above transcribe the corrected code; the superseded (buggy) double-well discretization is recorded in Remark 4.2 and audited in `docs/reference/phase1_energy_discretization_bug.md`.

## References


[1] Beniamin Bogosel and Édouard Oudet. Partitions of minimal length on manifolds. *Experimental Mathematics*, 26(4):496–508, 2017. YEAR CORRECTED 2026-08-21: this entry read year 2023 with no volume or pages, and its key was `bogosel2023partitions`. Crossref gives print publication 2 October 2017 (online 4 October 2016), Experimental Mathematics 26(4):496–508. This is the foundational reference for the Phase 1 relaxation and Phase 2 refinement methodology.

[2] Luciano Modica. The gradient theory of phase transitions and the minimal interface criterion. *Archive for Rational Mechanics and Analysis*, 98(2):123–142, 1987. Classical $\Gamma$-convergence

result for the Modica–Mortola functional; source of the interface constant $c_W = 2\int_0^1 \sqrt{W}$ used in the Phase 1 energy.